

NOEMA TELEKOM: A Consent-Gated File-Packet Communication Layer for Portable Agent Continuity

Adrian Lipa
Independent Researcher

Draft v0.1-alpha, May 2026

Abstract

This paper presents NOEMA TELEKOM, an experimental communication scaffold built on the NOEMA File-Packet Handshake model. The design targets local agents, portable memory runtimes, and consent-governed signaling without requiring an always-open listener as the primary handshake surface. Instead of beginning with a socket, a connection begins with a hash-verifiable file-packet, an append-only audit trace, an acknowledgement artifact, and an optional policy gate. The protocol separates transport artifacts from event logs, immutable state commitments, and latest-state pointers: `packet` $\neq$ `log` $\neq$ `snap` $\neq$ `CURRENT`. This separation is used to reduce false promotion of state, prevent silent overwrite of history, and make replay, consent bypass, and fake verification easier to detect. NOEMA TELEKOM extends this base model with anonymized addressing, call-state signaling, explicit accept/decline semantics, and policy files for security, privacy, consent, and key handling. A public-safe reference scaffold was prepared with test credentials and private keys removed. Local diagnostic tests show high performance for atomic rename operations, small fsync operations, JSONL append, hash verification, and compression of highly structured semantic data. The current release is a draft protocol and research artifact, not a production-certified security system and not a replacement for TLS or operating-system controls.

Keywords. offline-first protocols; file-packet handshake; local agents; append-only audit; consent gates; portable runtime state; anonymized addressing; semantic continuity.

1 Introduction

Modern agent systems increasingly require communication patterns that are portable, auditable, and safe under partial connectivity. The default engineering instinct is to expose a service endpoint and then secure it. This approach is powerful, but it also creates an attack surface: an open listener, an access-control boundary, a live authorization path, and a persistent runtime that must be monitored.

NOEMA TELEKOM explores a different starting point. The base handshake is listenerless: communication begins as an immutable file-packet dropped into a known inbox or exported as a portable artifact. The receiving side verifies hashes, validates state transitions, checks policy, and produces an acknowledgement or denial artifact. Only after explicit consent and policy validation may a higher-level runtime enter an active state.

The system is motivated by three requirements. First, agent-to-agent communication should be possible in controlled local or offline environments without pretending that a network channel is active. Second, every meaningful state change should leave an append-only, hash-verifiable trace. Third, a communication request must be semantically different from a coercive connection. In NOEMA TELEKOM, an incoming call is a request, not a force-call.

The contribution of this paper is a compact specification and analysis of NOEMA TELEKOM as a consent-gated layer over NOEMA File-Packet Handshake. The paper focuses on the

technical model: packet structure, state machine, audit semantics, anonymized addressing, privacy constraints, implementation scaffold, and experimental diagnostics.

2 Design Principles

2.1 Listenerless first contact

The primary handshake does not require a daemon to listen on a network port. A sender creates a file-packet containing a declared transition, sender and receiver semantic identifiers, a connection identifier, timestamps, payload metadata, boundary declarations, and hash-verifiable commitments. The receiver does not need to trust the sender’s narrative; it verifies the artifact.

2.2 Separation of state roles

The central invariant is

$$\text{packet} \neq \text{log} \neq \text{snap} \neq \text{CURRENT}. \quad (1)$$

A **packet** is an append-unique transport artifact. A **log** is an append-only event trace. A **snap** is an immutable historical state commitment. A **CURRENT** file is an overwriteable pointer to the latest accepted state, not mutable history. This separation prevents a latest-state update from being confused with a rewrite of memory.

2.3 No fake promotion

A packet must not claim encryption, anonymity, active connectivity, or canonical state unless the corresponding mechanism is implemented and verified. This rule is intentionally conservative. It protects the protocol from the common failure mode in which a prototype marks a transition as valid because the expected story was plausible.

2.4 Consent as a protocol primitive

NOEMA TELEKOM treats consent as part of the state machine rather than as an external user-interface convention. The recipient may accept, decline, ignore, or end a call. The state **RINGING** is a request, not a connection.

3 System Model

A NOEMA-compatible runtime consists of addressable entities called **ResEnts**. A **ResEnt** may be a user, an agent, a local process, an application, a model instance, or a service. Each **ResEnt** can emit or receive packets, produce audit events, and participate in state transitions.

Table 1: Core objects in the NOEMA File-Packet Handshake model.

Object	Role	Mutability
packet	Transport artifact	Append-unique
Log	Event trace	Append-only
snap	Historical state commitment	Immutable
CURRENT	Latest-state pointer	Overwriteable pointer
ACK	Verification/receipt artifact	Append-unique
Proof packet	Evidence bundle	Append-unique
Gate	Consent/policy transition	Audited transition

The base packet schema contains at least a protocol name and version, a transition name, a state name, a UTC timestamp, a connection identifier, sender and receiver semantic identifiers, a payload object, and boundary declarations. The protocol uses RFC 2119 terminology for requirement levels [1].

4 Protocol State Machine

The nominal state machine is shown in Figure 1. A candidate path is allowed when the receiver can verify local evidence but cannot prove a committed **CURRENT** update. This distinction is essential for portable runtime imports and cross-runtime proof packages.

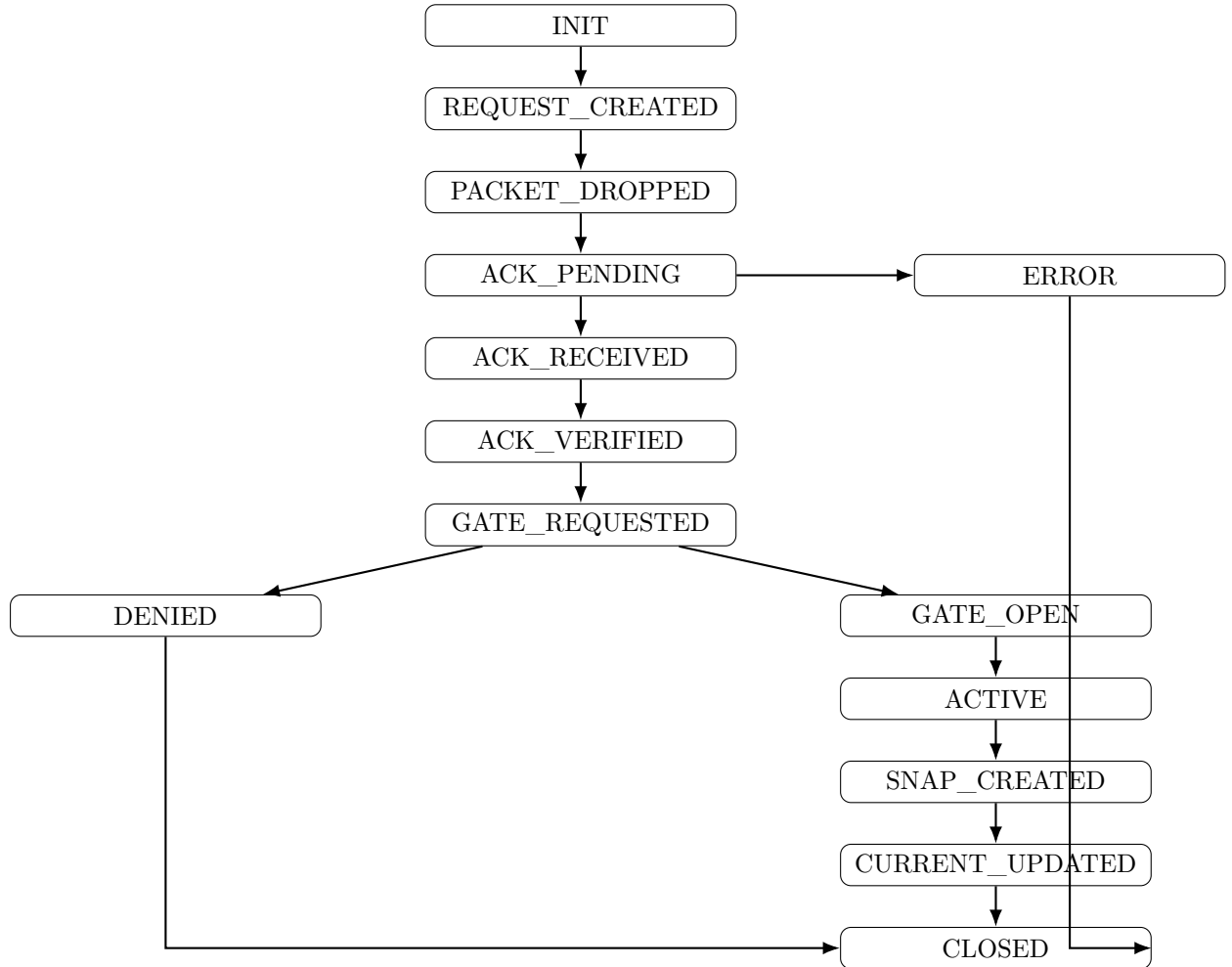


Figure 1: Nominal state machine for the listenerless handshake and consent-gated activation. Failure and denial paths produce audit events and terminate without silent promotion.

Invalid transitions include moving from **PACKET_DROPPED** to **ACTIVE** without **ACK** and gate evidence, updating **CURRENT** without **ACK** verification, or bypassing a consent requirement.

5 NOEMA Telekom

5.1 Purpose

NOEMA TELEKOM is a desktop-first scaffold for a consent-based NOEMA communication layer. It defines anonymized addressing, registries, call signaling, display surfaces for incoming

calls, policy files, and a reference Python package. The current scaffold does not start a main NOEMA server. Its purpose is to make the protocol inspectable before live deployment.

5.2 Addressing

The addressing layer defines semantic address forms such as

```
noema://agent/<anon_id>
noema://peer/<anon_id>
noema://subnet/<subnet_id>/<node_id>
noema://telekom/<network_id>/<anon_id>
```

The scaffold uses anonymized identifiers of the form

$$\text{anon_id} = \text{HMAC_SHA256}(\text{salt}, \text{semantic_address})[0 : 16]. \quad (2)$$

The registry stores `anon_id` values rather than raw local usernames or real identities. The salt is local secret material and must be excluded from exported registries. The 16-character truncation is suitable for a scaffold, but deployments requiring stronger collision resistance should increase this parameter.

5.3 Call semantics

The call lifecycle is

$$\text{IDLE} \rightarrow \text{RINGING} \rightarrow \{\text{ACCEPTED}, \text{DECLINED}, \text{MISSED}\} \rightarrow \text{ENDED}. \quad (3)$$

The state `RINGING` means request visibility. `ACCEPTED` is explicit consent. `DECLINED`, `MISSED`, and `ENDED` are valid non-consent or termination states. An administrative role may provide governance and routing authority, but not coercive access.

6 Security and Privacy Analysis

6.1 Threat model

The v0.1 threat model includes accidental corruption, malicious packet injection, replay, unauthorized `CURRENT` replacement, hash mismatch attacks, credential leakage, confused-deputy behavior, fake ACK/proof packets, social-engineered consent, and unsafe publication of test secrets. This scope is narrower than a full Internet security model and broader than a simple local file format.

The model follows a conservative position influenced by distributed-systems ordering concerns [5], protocol adversary models [6], and hash-linked commitment structures related to Merkle trees [7]. Unlike TLS [4], this draft does not claim to provide authenticated encrypted transport by itself. If encryption is configured, the implementation must use a real cryptographic provider and fail closed when it is unavailable.

6.2 Main mitigations

The primary mitigations are shown in Table 2.

6.3 Explicit non-claims

For public credibility, v0.1 explicitly does not claim guaranteed anonymity, guaranteed encryption, quantum communication, physics-breaking communication, trusted execution, automatic identity truth, global consensus, or active external binding without evidence. This paper analyzes the scaffold as an offline-first portable communication protocol, not as a replacement for Internet Protocol [2], TCP [3], TLS, operating-system security, or formal anonymity systems.

Table 2: Risk and mitigation summary.

Risk	Mitigation
Fake PASS	Byte-level verification, manifests, explicit verification status.
CURRENT hijack	Append-only snaps, hash-anchored pointer updates, audit trail, rollback.
Replay	Nonce, timestamp, connection identifier, replay cache, state-machine validation.
Secret publication	Public-safe repack, secret scan, manual review, no private keys in release.
Consent bypass	Consent gates, policy packets, deny-by-default transition logic.
Credential leakage	Separation of public scaffolds from local secret material.

7 Reference Scaffold

The public-safe scaffold contains policy documents, JSON schemas, display templates, a user registry template, call signaling examples, audit helpers, routing code, addressing code, and self-tests. Test credentials and private keys were removed from the publication package. The resulting repository package reports a secret scan status of **PASS**. The scaffold is intentionally small so that the state machine and policy boundaries can be reviewed before adding live services.

The recommended repository structure is:

```
docs/                protocol documents
reference/noema-telekom-scaffold-v0.1/ public-safe scaffold
artifacts/test-results/ local diagnostics
manifests/           machine-readable manifests
SHA256SUMS.txt       repository file hashes
```

8 Experimental Diagnostics

The local tests were executed in a controlled runtime directory and should be interpreted as implementation diagnostics, not as claims about global network throughput or production deployment. The tests measure file-system behavior relevant to the file-packet model: atomic rename, fsync, JSONL append, JSON parsing, hashing, copy throughput, and compression of structured versus random data.

Compression tests show the expected distinction between structured semantic data and random data. A 64 MiB semantic JSONL corpus compressed with 99.6116% space saving, while 64 MiB of random data expanded by approximately 0.0307%. A 128 MiB zero-filled corpus compressed with 99.9027% space saving. These results support the design intuition that semantic trace formats can be compact and fast to verify, but they do not imply compression of incompressible data.

9 Discussion

The main technical value of NOEMA TELEKOM is not raw throughput. It is the combination of listenerless first contact, consent-gated activation, append-only audit, and strict separation between packets, logs, snaps, and pointers. This combination makes the system resistant to a class of semantic failures common in agent prototypes: a request being treated as permission,

Table 3: Selected local diagnostic results.

Metric	Result
Sequential random write, 512 MiB	103.19 MiB/s
Sequential random read, 512 MiB	2104.02 MiB/s
Sequential random copy, 512 MiB	352.74 MiB/s
SHA-256 scan, 512 MiB	294.57 MiB/s
Small-file write, 2000 files of 4 KiB	3950 files/s
Small-file read+hash, 2000 files of 4 KiB	7634 files/s
JSONL append, 100k events	141277 events/s
Atomic rename, 1000 operations	2813 ops/s; p95 0.594 ms
fsync, 200 writes of 4 KiB	5713 ops/s; p95 0.298 ms
Semantic JSONL parse, 64 MiB	49.34 MiB/s

a pointer being treated as history, or a local candidate being promoted to a committed state without evidence.

The design also provides an economic and governance advantage. A protocol specification can be published openly to encourage interoperability, while official runtimes, certified nodes, brand usage, enterprise tooling, and hosted synchronization layers can remain governed by separate licensing and certification policies.

The primary limitation is that v0.1 is not a complete security protocol. Authentication, encryption, remote attestation, key rotation, multi-party consensus, and production deployment hardening remain future work. The reference scaffold is intentionally explicit about this limitation.

10 Conclusion

NOEMA TELEKOM demonstrates a consent-first communication layer for NOEMA-compatible agents and portable runtimes. It replaces the assumption of an always-open listener with a verifiable file-packet handshake, separates latest-state pointers from immutable history, and elevates consent and audit to protocol-level objects. The resulting scaffold is suitable for research, public review, and controlled local experimentation. Further work should formalize the cryptographic envelope, expand test vectors, add interoperability suites, and define certification levels for compliant nodes.

A Minimal Packet Example

```
{
  "protocol": "NOEMA_PROXY_CONNECT_REQUEST/0.1",
  "transition": "PROXY_CONNECT_REQUEST",
  "state": "PACKET_DROPPED",
  "created_utc": "2026-05-18T02:24:16Z",
  "connection_id": "noema:entity:connection-request:<nonce>",
  "from_semantic_id": "NOEMA:ENTITY_A:SESSION",
  "to_semantic_id": "NOEMA:ENTITY_B:PROXY",
  "payload": {},
  "boundaries": {
    "no_tcp": true,
    "no_server": true,
    "network_used": false
  }
}
```

}

B Publication Checklist

Before public release, a package should remove private keys and credentials, verify ZIP integrity, verify SHA256SUMS, include a license, include a threat model, include security boundaries, include a state machine, include explicit non-claims, add a version tag, publish source hashes, and run a secret scan.

References

- [1] S. Bradner. Key words for use in RFCs to Indicate Requirement Levels. RFC 2119, 1997.
- [2] J. Postel. Internet Protocol. RFC 791, 1981.
- [3] J. Postel. Transmission Control Protocol. RFC 793, 1981.
- [4] E. Rescorla. The Transport Layer Security (TLS) Protocol Version 1.3. RFC 8446, 2018.
- [5] L. Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978.
- [6] D. Dolev and A. C. Yao. On the security of public key protocols. *IEEE Transactions on Information Theory*, 29(2):198–208, 1983.
- [7] R. C. Merkle. A digital signature based on a conventional encryption function. In *Advances in Cryptology – CRYPTO ’87*, pages 369–378. Springer, 1988.